

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ, СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**КЛИЕНТСКА БИБЛИОТЕКА ЗА ДОСТЪП ДО
БАЗИ ДАННИ ЧРЕЗ ПРЯКА РЕАЛИЗАЦИЯ НА
МРЕЖОВИТЕ ПРОТОКОЛИ НА POSTGRESQL
И MYSQL**

КУРСОВ ПРОЕКТ

по дисциплина „Надграждащи технологии за бази данни“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Надежда Кучмова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Стандартът SQL и мястото на клиентския интерфейс в него	3
1.2. Технологии за достъп, изградени върху стандарта	3
1.3. Мрежовите протоколи на PostgreSQL и MySQL	4
1.4. Изводи от прегледа	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Разглеждани алтернативи	5
2.2. Критерии за сравнение	5
2.3. Обосновка на избора	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
3.1. Слоеви и отговорности	7
3.2. Протоколни кодеци	8
3.3. Асинхронен модел	9
3.4. Декодиране без копие	10
3.5. Пул от връзки и кеш на подготвените заявления	10
3.6. Слой за типово изображение	11
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	12
4.1. Използвани средства	12
4.2. Структура на изходния текст	12
4.3. Ключови места в реализацията	13
4.4. Обработка на грешките	16
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	17
5.1. Какво се проверява	17
5.2. Еталони и натоварване	17
5.3. Среда и възпроизводимост	18
5.4. Брой повторения и отхвърляне	18

5.5. Показатели	19
5.6. Условия за валидност	19
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	20
6.1. Проверка на коректността	20
6.2. Закъснителност на едно заявление	20
6.3. Обръщания по мрежата и брой съобщения	21
6.4. Текстов срещу двоичен формат	22
6.5. Закъснителност на заемане от пул	23
6.6. Анализ	23
ЗАКЛЮЧЕНИЕ	25
ЛИТЕРАТУРА	27
ПРИЛОЖЕНИЯ	29

УВОД

Достъпът до релационна база данни от програма на С или С++ почти винаги минава през чужд слой. Приложението извиква клиентска библиотека на доставчика, тя от своя страна говори с процеса на сървъра по мрежов протокол, а между двете нерядко стои и диспечер на драйвери като ODBC. Всеки слой има своя цена: собствен модел на паметта, собствено разбиране за типовете, собствено копие на резултатните редове и собствен, обикновено синхронен, модел на изчакване. Приложението плаща тази цена, без да я вижда, защото слоят е даденост.

Настоящият курсов проект разглежда алтернативата. Мрежовите протоколи на PostgreSQL и на MySQL са публично специфицирани [1, 2], което означава, че клиентът може да бъде написан срещу самата спецификация, а не срещу чужда библиотека. Този избор премахва междинните слоеве и прави видими решения, които иначе са скрити: къде се копира буфер, кога се сериализира стойност, колко обръщания до сървъра струва едно подготвено заявление. Същият избор носи и очевидни рискове, които проектът е длъжен да отчете честно, а не да премълчи.

Целта на проекта е да се проектира и реализира клиентска библиотека на С++20, която реализира пряко мрежовите протоколи на PostgreSQL и MySQL, и да се измери поведението ѝ спрямо доставчиковите клиентски библиотеки върху едно и също натоварване.

За постигането на целта са формулирани следните **задачи**:

1. преглед на стандарта SQL и на изградените върху него технологии за достъп, както и на спецификациите на двата разглеждани мрежови протокола;
2. анализ на възможните решения, слоест диспечер на драйвери, доставчикова клиентска библиотека или пряка реализация на протокола, и обосновка на избора;
3. проектиране на слоеста архитектура с ясно отделени транспорт, протоколни кодеци, декодер на редове, пул от връзки и слой за типово изображение;
4. програмна реализация на архитектурата за двата протокола;
5. изграждане на методика за измерване и на възпроизводима среда за нея;
6. експериментално сравнение с `libpq` и `libmysqlclient` и анализ на получените резултати.

Обхватът на проекта е клиентската страна на комуникацията: установяване на връзка, удостоверяване, изпълнение на прости и на подготвени заявления, декодиране на резултатни редове и управление на пула от връзки. Извън обхвата остават администрирането на сървъра, репликацията, разпределените транзакции и всяка форма на съхранение от страна на клиента.

Структура на изложението. Раздел I представя предметната област и прегледа на литературата. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III описва проектирането, а Раздел IV програмната реализация. Раздел V дефинира методиката за измерване, а Раздел VI представя резултатите и техния анализ. Заключението обобщава постигнатото и посочва посоките за продължение.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Стандартът SQL и мястото на клиентския интерфейс в него

Стандартът ISO/IEC 9075 описва не само езика за заявки, но и начина, по който външна програма стига до данните. Част 1 задава общата рамка и понятията, с които останалите части си служат [3]. Част 3 определя *интерфейса на ниво извикване* (Call-Level Interface, CLI), тоест процедурен интерфейс, чрез който приложение на общоцелеви език изпраща заявления и получава резултати, без тези заявления да са вградени в кода при компилация [4]. Именно тази част е шаблонът, върху който стъпват практическите технологии за достъп.

Разграничението, което стандартът прави, е съществено за целия проект. Стандартът нормира *интерфейса*, който приложението вижда, но не нормира *протокола*, по който клиентът и сървърът си говорят по мрежата. Протоколът е дело на всеки доставчик поотделно. Следователно преносимостта, която CLI обещава, се постига винаги чрез междинен слой, който превежда стандартните извиквания към нестандартния протокол на конкретния сървър.

1.2. Технологии за достъп, изградени върху стандарта

ODBC е най-разпространената реализация на идеята за CLI. Приложението се свързва с *диспечер на драйвери*, който зарежда драйвер за конкретния източник на данни и пренасочва извикванията към него [5]. Върху този модел исторически се надграждат OLE DB, ADO и по-късно ADO.NET, които добавят обектен модел, работа с несвързани множества от редове и по-богата типова система [6]. Общото между всички тях е броят на границите, през които минава един резултатен ред, преди да стигне до приложението.

Отделно от релационния път стоят спецификациите за слабо структурирани данни и за директорийни услуги. XML остава нормативната основа за обмен на документно структурирани данни между разнородни системи [7], а LDAP описва достъпа до йерархични директорийни хранилища с протокол, който по устройство е близък до разглежданите тук: двоичен обмен на съобщения с ясно определени етикети и дължини, върху постоянна връзка [8]. Сравнението с LDAP е полезно, защото показва, че прякото

говорене по протокол е обичайна практика извън релационния свят.

1.3. Мрежовите протоколи на PostgreSQL и MySQL

Протоколът на PostgreSQL е съобщително ориентиран: всяко съобщение носи еднобайтов етикет, дължина и тяло, а фазите на работа са ясно отделени, стартиране и удостоверяване, прост цикъл на заявка, разширен цикъл с Parse/Bind/Execute за подготвени заявления [1]. Протоколът на MySQL е организиран около пакети с последователен номер и разчита в по-голяма степен на позиционно кодиране и на цели числа с променлива дължина [2]. Двата протокола решават една и съща задача с различни средства, което ги прави удобна двойка за общ клиентски слой.

Референтните клиентски библиотеки на двата доставчика са добре документирани и служат за еталон в тази работа. Документацията на libpq описва подробно както интерфейса, така и семантиката на асинхронния режим [9]. По-общата литература за системи за обработка на транзакции обяснява защо клиентският слой не е неутрален към производителността: границите на транзакциите, изчакванията и броят обръщания по мрежата определят наблюдаваната закъснителност повече от скоростта на самия сървър [10, 11].

1.4. Изводи от прегледа

Прегледът очертава три нива, на които може да се стъпи: стандартизиран интерфейс с диспечер на драйвери, доставчикова клиентска библиотека и самата спецификация на протокола. Всяко следващо ниво надолу премахва слой и абстракция, но добавя отговорност за коректност. **[TODO: да се добавят два или три съпоставими академични или индустриални източника за измервана цена на слоевете при достъп до бази данни, ако такива бъдат намерени; ако не бъдат, това да се заяви изрично тук]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Разглеждани алтернативи

Разгледани са три решения на една и съща задача, а именно да се изпълни заявление от програма на C++ и да се получат резултатните редове в типизиран вид.

Алтернатива А, слоест диспечер на драйвери. Приложението извиква ODBC, а диспечерът зарежда драйвер за конкретния сървър [5]. Предимството е една входна точка за много източници на данни, включително такива, за които никой не би написал клиент сам. Недостатъкът е, че всеки резултатен ред минава през договорения от стандарта тип и през буфер, зададен от приложението предварително, а асинхронната работа не е част от модела.

Алтернатива Б, доставчикова клиентска библиотека. Приложението се свързва пряко с `libpq` или с `libmysqlclient`. Отпада един слой, а семантиката е точно тази на сървъра. Остават обаче два въпроса: библиотеките са на C и налагат собствено управление на паметта на границата, а асинхронните им режими се различават помежду си достатъчно, за да не се обединяват просто [9].

Алтернатива В, пряка реализация на мрежовия протокол. Приложението кодира и декодира съобщенията само, срещу публикуваната спецификация [1, 2]. Отпадат всички междинни слоеве, буферът за получаване се притежава от самата библиотека и декодирането може да работи върху него без копие. Цената е, че коректността вече е отговорност на проекта, включително частите на протокола, които се срещат рядко и затова се откриват късно.

2.2. Критерии за сравнение

Сравнението се води по критерии, всеки от които е проверим, а не въпрос на вкус: брой слоеве, през които минава един ред; брой копия на данните между сокета и типизираната стойност; наличие на естествен асинхронен модел; възможност за кеширане на подготвени заявления от страна на клиента; обем на кода, който проектът поема да поддържа; чувствителност към промени във версията на сървъра.

Таблица 1. Съпоставка на разгледаните алтернативи по приетите критерии

Критерий	А. ODBC	Б. Доставчикова	В. Пряк протокол
Слоеве до реда	[TODO: ?]	[TODO: ?]	3: транспорт, кодек, декодер
Копия на данните	[TODO: ?]	[TODO: ?]	0 до изгледа, 1 при поискано притежание
Асинхронен модел	не по спецификация [5]	[TODO: ?]	да, съпрограми на C++20
Кеш на заявленията	[TODO: ?]	[TODO: ?]	да, по връзка, 32 записа по подразбиране
Поддържан код	[TODO: ?]	[TODO: ?]	4558 реда в include/ и src/

Колоната за алтернатива В (Табл. 1) е попълнена, след като реализацията приключи, и числата в нея са преброени в самото хранилище. Колоните за А и Б остават незапълнени умишлено: броят слоеве и броят копия в чужда реализация не се установява без четене на нейния изходен текст, а такова четене не е правено. Празната клетка е по-полезна от правдоподобно предположение, което после ще бъде цитирано като измерване. **[TODO: колони А и Б, след преброяване в изходния текст на съответните реализации]**

2.3. Обосновка на избора

Проектът избира алтернатива В. Основанието не е производителност сама по себе си, а това, че само тя позволява цената на достъпа да бъде наблюдавана. При А и Б броят копия и обръщания е скрит зад чужд код и може да се оцени само косвено. При В всяко копие е ред в собствения изходен текст и следователно може да бъде премахнато или защитено с аргумент. Дисциплината изисква и сравнение между слоевия модел и прякото говорене по протокол, което тази постановка прави естествено.

Изборът се приема заедно с цената му. Проектът поема поддръжката на две независими кодиращи схеми, поема риска от разминаване с бъдещи версии на сървърите и се отказва от преносимостта към източници на данни извън двата разглеждани. Тези ограничения са записани изрично, за да бъдат отчетени в заключението, а не преоткрити там.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Слоевете и отговорности

Системата е разделена на пет слоя с еднопосочна зависимост, отдолу нагоре. Най-отдолу стои *транспортът*, който притежава сокета и буферите и не знае нищо за съдържанието на байтовете. Над него стои *протоколният кодек*, по един за PostgreSQL и за MySQL, който превръща поток от байтове в съобщения и обратно. Следва *декодерът на редове*, който превръща тялото на едно съобщение с данни в изгледи към стойности, без да ги копира. Над него е *пултът от връзки*, който управлява живота на връзките, а всяка връзка носи собствен кеш на подготвените заявления. Най-отгоре стои *слоят за типово изображение*, който свързва стойностите от протокола с типовете на C++.

Границата между всеки два слоя е тесен договор, а не общ обект със състояние. Целта е всеки слой да се тества самостоятелно: кодекът се захранва със записан байтов поток, декодерът с готово тяло на съобщение, пултът с фиктивна връзка. Това разделение е и условието за смисленото сравнение с доставчиковите библиотеки, защото позволява цената да се отнесе към конкретен слой. От 85-те модулни теста нито един не изисква работещ сървър (Табл. 2).

Таблица 2. Слоевете, файловете и начин на самостоятелна проверка

Слой	Файлове	Как се проверява без сървър
Транспорт	net.hpp, net.cpp, task.hpp	двойка свързани сокета върху 127.0.0.1, отворена от самия тест
Протоколен кодек	pg/protocol.hpp, mysql/protocol.hpp	байтови потоци, съставени поле по поле от спецификацията
Декодер на редове	value.hpp, value.cpp	изкуствени field_view в двата формата
Пул и кеш	pool.hpp, lru_cache.hpp	фиктивна връзка, която удовлетворява само изискването poolable
Типово изображение	row_map.hpp	списък от колони с имена, без протокол

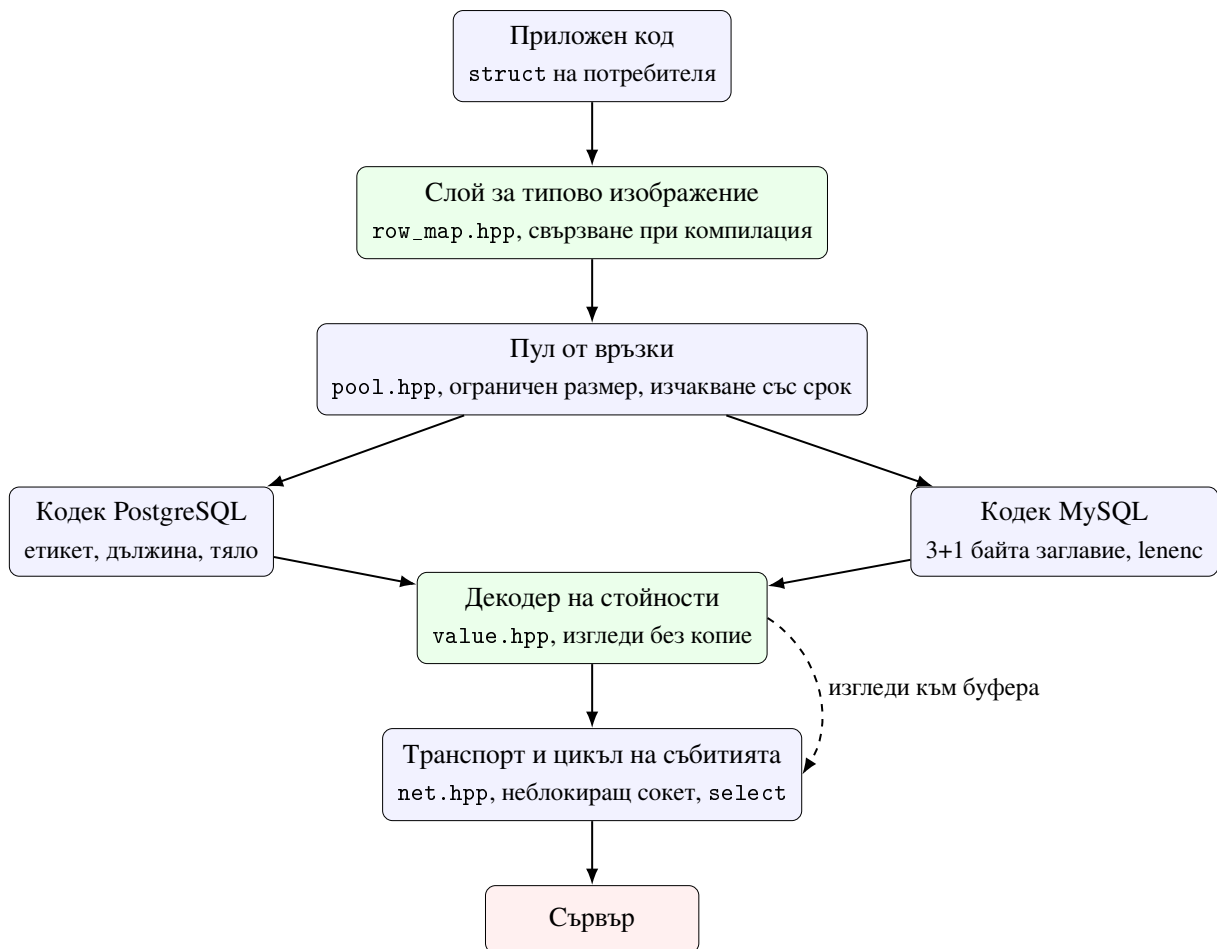
3.2. Протоколни кодеци

Кодекът на PostgreSQL следва съобщителния модел на протокола: етикет, дължина, тяло, и двата цикъла на изпълнение, простия и разширения [1]. Кодекът на MySQL следва пакетния модел с номер на последователност и цели числа с променлива дължина [2]. Двата кодека нямат общ базов клас. Общото между тях е изразено като изискване към типа, тоест като concept на име `frame_codec`:

```
1 template <class C>
2 concept frame_codec = requires(byte_span in, const typename C::frame& f) {
3     typename C::frame;
4     { C::peek(in) } -> std::same_as<std::optional<typename C::frame>>;
5     { f.consumed } -> std::convertible_to<std::size_t>;
6     { C::describe(f) } -> std::convertible_to<const char*>;
7 };
```

Листинг 1: Общото между двата кодека, изразено като изискване към типа

Изискването е проверимо, а не декоративно: в края на всеки от двата заглавни файла стои `static_assert(frame_codec<codec>)`, така че кодек, който престане да удовлетворява договора, не се компилира. Ключовата дума в договора е `peek`: функцията *разглежда*, но не консумира. Непълно съобщение струва едно връщане на празна стойност, никога загубен байт.



Фигура 1. Слоевете и посоката на зависимостите. Плътните стрелки са извиквания, прекъснатата е препратка към чужда памет: декодираните стойности са изглед към получаващия буфер на транспорта

3.3. Асинхронен модел

Всяка операция от свързването нагоре е съпрограма от типа `task<T>`. Типът е *мързелив*: създаването му не стартира нищо, а изчакването му предава управлението на извиканата съпрограма чрез симетричен трансфер. Разликата не е стилистична. При верига от N изчаквания симетричният трансфер струва едно възобновяване, докато наивната реализация, която възобновява откъдето на `await_suspend`, натрупва N рамки в стека на нишката.

Цикълът на събитията е един на нишка и не съдържа нито един ключ за заключване. Готовността на сокетите се проверява със `select`, защото това е единственото повикване с еднакво поведение на Windows и на POSIX; таванът от `FD_SETSIZE` дескриптора е записан в изходния текст като известно ограничение заедно с посоката на надграждане.

3.4. Декодиране без копие

Резултатният ред пристига като част от получаващия буфер, притежаван от транспорта. Декодерът не изгражда нови обекти за стойностите, а издава изгледи от вида `std::span<const std::byte>` и `std::string_view` към този буфер. Копие се прави само когато приложението изрично поиска притежание на стойността, тоест когато поиска `std::string` вместо `std::string_view`.

Условието за коректност е част от публичния договор: изгледите са валидни само докато трае извикването на обработчика на реда. Механизмът, който прави това вярно, е в самия буфер. Получаващият буфер `recv_buffer` премества байтове най-много веднъж на уплътняване, а уплътняване се прави само при четене от сокета. Следователно между две четения нищо не помръдва и изгледите остават валидни. Тестът `recv_buffer_keeps_views_stable_until_consumed` проверява точно това твърдение, като сравнява адреса на издадения изглед преди и след добавяне на нови байтове.

3.5. Пул от връзки и кеш на подготвените заявления

Пулът е асинхронен и ограничен. Заявка за връзка, която не може да бъде удовлетворена веднага, спира изпълнението на извикващата съпрограма, вместо да блокира нишка. Изчакването има срок, който се състезава със самото освобождаване: побеждава първото от двете събития, а губещото намира вече уредено състояние и не прави нищо.

Освободената връзка се предава *пряко* на най-дълго чакащия, а не се връща в списъка на свободните. Разликата изглежда несъществена и не е. Мерките в Раздел VI са направени и по двата начина: при връщане през списъка съпрограмата, която освобождава връзката, продължава да се изпълнява преди събудения чакащ и си взема същата връзка обратно, така че чакащият гладува.

Всяка връзка носи собствен кеш на подготвените заявления, защото подготвеното заявление в двата разглеждани протокола е свързано със сесията, а не с базата. Кешът е с ограничен размер и с политика на изместване на най-отдавна използвания запис. Измественият запис се *връща* на извикващия, а не се изхвърля мълчаливо: заявлението продължава да съществува на сървъра и трябва да бъде затворено със съобщение `Close`. Подразбиращият се размер е 32 записа на връзка.

3.6. Слой за типово изображение

Слоят превръща стойност от протокола в типизирана стойност на C++. Съответствието се задава декларативно, като `constexpr` наредена n -торка от указатели към членове, и се проверява при компилация: член, чийто тип няма декодер, не се компилира, а колона, която липсва в резултата, се съобщава по име при изграждането на изображението, веднъж за целия резултатен набор, а не веднъж на ред.

Съответствието между типовете е дадено в Табл. 3. Случаите без точно съответствие са три и всеки е решен явно. Числото с произволна точност се предава като десетичен текст, защото превръщането му в `double` губи точността, заради която типът съществува. Празната стойност се чете само през `std::optional`; всеки друг избор изисква да се измисли стойност за нея. Масивът се декодира само едномерен, а вложеният масив се съобщава като грешка, вместо да бъде сплескан.

Таблица 3. Съответствие между типовете на двата сървъра и типовете на C++

PostgreSQL	MySQL	C++	Бележка
bool	tinyint(1)	bool	двоично един байт, текстово t/f/0/1
int2, int4, int8	smallint ... bigint	int16_t ... int64_t	двоично с проверка за препълване
float4, float8	float, double	float, double	двоично IEEE 754, мрежов ред
text, varchar	varchar, char	string_view	без копие; <code>std::string</code> прави копието явно
bytea	blob	vector<byte>	текстово в шестнадесетичен и в осмичен запис
timestamp	datetime	timestamp	двоично микросекунди от 2000-01-01
numeric	decimal	std::string	точен десетичен запис, без загуба само едномерни
int4[] и подобни	няма	vector<T>	
всяка колона с NULL	същото	optional<T>	единственият коректен прочит

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Използвани средства

Реализацията е на C++20 [12]. От стандарта се използват три възможности, всяка с конкретно основание. Съпрограмите носят асинхронния модел, без да налагат обратни извиквания и без да раздробяват логиката на протокола между функции. Изискванията към типовете, тоест concept, изразяват общото между двата кодека без виртуални функции. `std::span` и `std::string_view` са носителят на декодирането без копие.

Сглобяването е с CMake. Съществено решение е, че проектът няма нито една задължителна външна зависимост: на чиста машина са нужни само компилатор за C++20 и CMake, а при конфигуриране не се тегли нищо от мрежата. Това има цена и тя е платена на три места. Тестовата рамка е собствена, 150 реда в `tests/check.hpp`, и се свързва с CTest чрез `add_test`. Измерването е със собствен часовник върху `std::chrono::steady_clock`. Хеш функциите, от които зависят двата обмена за удостоверяване, са реализирани в `src/crypto.cpp`, вместо да се вземат от криптографска библиотека: MD5 [13], SHA-1 и SHA-256 [14], HMAC [15] и PBKDF2. Всяка от тях е проверена срещу публикуваните контролни вектори на съответния документ, а не срещу собствения си изход.

Единствената незадължителна зависимост е `libpq`, търсена с `find_package(PostgreSQL QUIET)` и използвана само от измервателната програма, която съобщава изрично, когато я няма. Интеграционните тестове изискват работещи сървъри и затова средата се вдига с Docker Compose, което прави версиите на сървърите част от конфигурацията и следователно записани: PostgreSQL 16.4 и MySQL 8.0.39.

4.2. Структура на изходния текст

Изходният текст е разделен по слоевете от Раздел III. Публичните заглавни файлове са в `include/conduit/`, реализацията в `src/`, тестовете в `tests/`, измервателната програма в `benchmarks/`, показната програма в `examples/`. Двата протокола са в отделни поддървета, които не се включват взаимно.

1	<code>include/conduit/</code>	
2	<code>bytes.hpp</code>	байтови четци и писачи, получаващ буфер
3	<code>task.hpp</code>	<code>task<T></code> , симетричен трансфер, <code>detached_task</code>
4	<code>net.hpp</code>	сокет, цикъл на събитията, изчакване със срок
5	<code>crypto.hpp</code>	MD5, SHA-1, SHA-256, HMAC, PBKDF2, base64

6	scram.hpp	клиентска страна на SCRAM-SHA-256
7	codec.hpp	concept frame_codec, общото между кодеците
8	value.hpp	field_view и декодерите по тип на C++
9	trace.hpp	проследяване на протоколните съобщения
10	lru_cache.hpp	кеш с изместване, връща измествения запис
11	row_map.hpp	типово изображение със свързване при компилация
12	pool.hpp	ограничен асинхронен пул от връзки
13	pg/protocol.hpp	съобщения на PostgreSQL, кодиране и декодиране
14	pg/connection.hpp	прост и разширен цикъл, кеш на заявленията
15	mysql/protocol.hpp	пакети, ръкостискане, текстов протокол
16	mysql/connection.hpp	ръкостискане, COM_QUERY, COM_PING
17	src/	реализация на изброените заглавни файлове
18	tests/	85 модулни случая и 12 интеграционни
19	benchmarks/bench.cpp	измервателната програма
20	examples/demo.cpp	показната програма
21	docker-compose.yml	PostgreSQL 16.4 и MySQL 8.0.39

Листинг 2: Дърво на изходния текст

4.3. Ключови места в реализацията

4.3.1. Разчитане на непълно съобщение

Транспортът може да върне произволна част от пакет, така че кодекът трябва да е способен да спре и да продължи, без да губи състояние. Решението е кодекът да няма състояние изобщо. Функцията `peek_frame` гледа буфера и или връща цяло съобщение, или връща празна стойност; тя не консумира нищо. Консумирането е отделна стъпка, която връзката прави след като е обработила съобщението.

```

1 std::optional<frame> peek_frame(byte_span in) {
2     if (in.size() < 5) return std::nullopt;
3     std::uint32_t len = load_be32(in.data() + 1);
4     if (len < 4) throw protocol_error("message length below the minimum of
↪ four");
5     std::size_t total = std::size_t{len} + 1;
6     if (in.size() < total) return std::nullopt;
7     frame f;
8     f.tag = static_cast<char>(std::to_integer<std::uint8_t>(in[0]));
9     f.body = in.subspan(5, total - 5);
10    f.consumed = total;
11    return f;
12 }
```

Листинг 3: Рамкиране на съобщение на PostgreSQL, без състояние и без консумиране

Цикълът, който я използва, е три реда и е единственото място, което говори със

сокета:

```
1 task<frame> connection::read_message() {  
2     for (;;) {  
3         auto f = peek_frame(buf_.readable());  
4         if (f) co_return *f;  
5         bool alive = co_await read_some(*loop_, sock_, buf_);  
6         if (!alive) {  
7             poisoned_ = true;  
8             throw io_error("PostgreSQL server closed the connection");  
9         }  
10    }  
11 }
```

Листинг 4: Четене на едно съобщение, с прекъсване при затворена връзка

Тестът `peek_frame_refuses_to_consume_a_partial_message` подава на кодека всяко възможно начало на едно съобщение, от нула до пълната дължина, и изисква да получи празна стойност за всяко от тях.

4.3.2. Грешка от сървъра в средата на резултатен поток

Грешка, съобщена от сървъра по правилата на протокола, задължава клиента да изчерпи обмена до синхронизиращата точка, преди връзката да бъде използвана отново. В разширения цикъл на PostgreSQL сървърът изхвърля всички съобщения до следващия Sync. Реализацията записва грешката, но *продължава* да чете до ReadyForQuery и чак тогава хвърля изключение. Затова връзка, върху която е изпълнено грешно заявление, остава годна: доказва го интеграционният тест `pg_live_server_error_leaves_the_connection_usable`, който след грешка с код 42P01 изпълнява ново заявление по същата връзка.

Разграничението между трите вида грешки е носено от типовете. `protocol_error` означава байтов поток, който не отговаря на спецификацията, и е неотстранима: позицията в потока е загубена. `io_error` е отказ на транспорта. `server_exception` носи полетата на съобщението за грешка и е единствената, след която връзката се използва пак. Изборът между изключения и тип със стойност или грешка е направен в полза на изключенията за първите две, защото и двете са наистина изключителни и всеки междинен слой би ги препращал нагоре без обработка; за очаквано отсъствие на стойност се използва `std::optional`, а не изключение.

4.3.3. Заемане на връзка от асинхронния пул

Заемането се състезава с изтичане на срок. Съпрограмата не може да изчаква едновременно свободна връзка и таймер, затова едното от двете идва като обратно извикване, което урежда другото. Общото състояние е споделено чрез `shared_ptr`, за да няма достъп до унищожена рамка, ако таймерът се задейства след събуждането.

```
1 void await_suspend(std::coroutine_handle< > h) {  
2     w->co = h;  
3     pool->waiters_.push_back(w);  
4     auto* loop = pool->loop_;  
5     auto shared = w;  
6     auto* p = pool;  
7     loop->call_at(deadline, [loop, shared, p]() {  
8         if (shared->settled) return;  
9         shared->settled = true;  
10        shared->timed_out = true;  
11        p->forget_waiter(shared);  
12        loop->schedule(shared->co);  
13    });  
14 }
```

Листинг 5: Изчакване за свободна връзка със срок

Освобождаването предава връзката пряко на чакащия. Първата реализация я връщаше в списъка на свободните и събуждаше чакащия, което изглежда еквивалентно. Измерването показва, че не е: закъснителността на заемане излизаше двумодална, под една микросекунда за 99 на сто от пробите и над 200 милисекунди за останалите. Причината е, че съпрограмата, която освобождава връзката, продължава да се изпълнява преди събудения чакащ и си взема същата връзка обратно. Поправката е петнадесет реда, а тестът `pool_hands_a_released_connection_straight_to_the_waiter` изисква два работника при пул с размер едно да се редуват, а не единият да завърши преди другия.

4.3.4. Кеш на подготвените заявления и неговото обезсилване

Кешът е по SQL текст и стойността е името на заявлението на сървъра заедно с описанието на колоните. При попадение реализацията не изпраща `Parse` и `Describe`, а взема описанието на колоните от кеша, защото сървърът вече няма да го изпрати.

Обезсилването има два случая. Изместване от кеша връща измествения запис, върху който се изпраща `Close` за заявлението; иначе всяко изместване би оставяло по едно заявление на сървъра. Вторият случай е разминаване: ако сървърът отговори с код 26000,

тоест не познава името на заявлението, което клиентът смята за кеширано, записът се изтрива и целият поток се изпълнява повторно веднъж. Това се случва след `DISCARD ALL` или след смяна на сървъра зад пулър и извикващият не бива да го вижда.

4.3.5. Удостоверяване

Реализирани са четири механизма за PostgreSQL: доверен, парола в чист вид, MD5 и SCRAM-SHA-256 [16, 17]. Последният не е разширение, а необходимост: PostgreSQL 14 и следващите го използват по подразбиране, така че без него клиентът не може да се свърже със сървър, който не е нарочно отслабен. Средата в `docker-compose.yml` е оставена с подразбиращия се `scram-sha-256` точно по тази причина.

Обменът е проверен срещу публикувания в RFC 7677 пример: при фиксиран клиентски случаен низ реализацията възпроизвежда буква по буква и клиентското доказателство, и подписа на сървъра. За MySQL е реализиран `mysql_native_password`, който е SHA-1 верига [14], включително отговорът на искане за смяна на приставката. Приставката `caching_sha2_password` не е реализирана и това е съобщено с ясна грешка, а не с мълчалив отказ.

4.4. Обработка на грешките

Грешките се разделят на три вида, вече изброени: протоколна грешка, грешка от сървъра и грешка на транспорта. Първите две се различават съществено, защото само втората оставя връзката годна за по-нататъшна употреба. Връзка, която е получила протоколна грешка или отказ на транспорта, се отбелязва като негодна чрез флага `poisoned_`; пулът проверява `is_usable()` и при връщане, и при заемане, така че такава връзка се изхвърля, а не се раздава повторно. Тестът `pool_discards_a_connection_the_server_closed_underneath_it` го проверява, като убива връзката, докато тя е заета.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверката има две отделни цели, които не бива да се смесват. Първата е коректност: реализацията трябва да чете и записва това, което спецификацията предписва, и да получава от сървъра същите стойности, които са били записани. Втората е цена: закъснителност, брой обръщания по мрежата и цена на декодирането. Коректността е предусловие. Измерване върху реализация, която дава различни резултати, няма смисъл.

Коректността се проверява на две нива. Модулните тестове сравняват изхода на кодеците с байтови последователности, съставени поле по поле от спецификацията, и с публикуваните контролни вектори за хеш функциите. Интеграционните тестове изпълняват заявления срещу работещи сървъри и сравняват прочетеното със записаното.

5.2. Еталони и натоварване

Първоначалният замисъл предвиждаше сравнение с `libpq` [9] и `libmysqlclient` върху едно и също натоварване. Измервателната програма съдържа такъв клон: при конфигуриране `CMake` търси `libpq` и, ако я намери, сглобява и еталонно измерване със същото заявление срещу същия сървър. На машината, на която са направени измерванията от Раздел VI, `libpq` не е инсталирана, което програмата съобщава изрично и не отчита никакво сравнение. Това е записано като липсващо, а не заобиколено с оценка.

Натоварването се състои от пет случая, всеки изпълнен от една и съща програма върху една и съща база:

1. едно заявление за един ред през простия цикъл;
2. същото заявление през разширения цикъл, при празен кеш на подготвените заявления;
3. същото заявление през разширения цикъл, при попадение в кеша;
4. голям резултат от 5000 реда с четири колони, веднъж в текстов и веднъж в двоичен формат;
5. заемане на връзка от пул при размер 1, 2, 4 и 8 и едновременност 1, 4 и 16.

Схемата на тестовата таблица е една и съща за двата сървъра до типовете, които всеки от тях именува различно, и е дадена в Приложение А.

5.3. Среда и възпроизводимост

Средата се вдига с Docker Compose [18], за да са фиксирани версиите на сървърите и началното състояние на базите. Конфигурацията, при която са направени измерванията в Раздел VI, е дадена в Табл. 4.

Таблица 4. Конфигурация на машината и на средата при измерването

Компонент	Стойност
Операционна система	Windows 11 Pro N, версия 10.0.26200
Процесор	12 логически ядра
Компилатор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Режим на сглобяване	CMAKE_BUILD_TYPE=Release, генератор Ninja 1.13.2
Стандарт	C++20, без разширения на компилатора
CMake	4.3.2
Docker	29.4.3, Compose v5.1.3
PostgreSQL	образ postgres:16.4, порт 55432, удостоверяване scram-sha-256
MySQL	образ mysql:8.0.39, порт 33306, приставка mysql_native_password
Мрежа	обратна връзка (127.0.0.1) към контейнерите на същата машина
Еталонни библиотеки	libpq и libmysqlclient не са налични на машината

Обстоятелството, че сървърите работят в контейнери на същата машина, а не на отделен хост, е съществено за тълкуването на числата и е отбелязано изрично: разсейването, което се вижда в Раздел VI, идва в голямата си част от планировчика на виртуализационния слой, а не от клиента.

5.4. Брой повторения и отхвърляне

Всеки случай от първите четири се изпълнява 520 пъти, от които първите 20 се отхвърлят като загряващи; за големия резултат повторенията са 120 при 20 загряващи, защото едно повторение чете 5000 реда. За заемането от пул всеки работник изпълнява 40 заемания, след като пулт е бил предварително напълнен с едновременни заемания, така че цената на свързването да не влезе в измерването на заемането.

Отчитат се медиана, 95-и процентил, средна стойност и извадково стандартно

отклонение. Средна стойност без разсейване не позволява да се съди дали разликата между два случая изобщо е значима, а при разпределение с тежка дясна опашка, каквото се наблюдава тук, медианата и средната стойност се различават съществено и двете носят различна информация.

Суровите проби от всяко повторение се записват във файл `benchmarks/results/raw_samples.csv` при подаден флаг `--raw`, откъдето са изведени всички таблици в Раздел VI. Извадка от този файл е в Приложение Г.

5.5. Показатели

Измерват се четири показателя. Закъснителност на едно логическо действие, по проценти на разпределението. Брой протоколни съобщения за едно логическо действие. Време за декодиране на един резултатен набор, отделено от общото време чрез отделен часовник в самия обработчик на ред. Закъснителност на заемане на връзка от пул.

Броят съобщения не е времеви и се получава чрез броене в самия клиент, което го прави независим от машината и затова най-удобен за сравнение между реализации. Броенето се прави от механизма за проследяване, който е част от библиотеката и е изключен по подразбиране.

5.6. Условия за валидност

Измерването се приема за валидно само ако са изпълнени следните условия: и двата случая изпълняват едно и също заявление върху една и съща база; сървърът е един и същ процес за всички случаи в един пуск; загряващите повторения са отхвърлени; и разсейването е отчетено. Първите две условия се осигуряват от структурата на измервателната програма, която прави цялата подготовка веднъж в началото на пуска и след това не пипа базата; последните две са в кода, който събира пробите. Измерване, което не отговаря на някое от условията, се записва като невалидно и не участва в изводите.

Едно условие от първоначалния замисъл отпада: съпадението байт по байт с еталонна библиотека не може да бъде проверено, защото еталонна библиотека липсва на машината. Затова изводите в Раздел VI сравняват случаи *в рамките на самата реализация* и не твърдят нищо за отношението към `libpq`.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Проверка на коректността

Проверката за коректност е предусловие за всичко останало и е изпълнена изцяло.

Таблица 5. Резултат от проверката за коректност

Набор	Случаи	Съвпадения	Разминавания
Модулни, без сървър	85	85	0
Интеграционни, PostgreSQL 16.4	6	6	0
Интеграционни, MySQL 8.0.39	6	6	0

Командите, които произвеждат Табл. 5: `cmake --build build`, след това `ctest --test-dir build -L unit` и `ctest --test-dir build -L integration` при вдигната среда. Модулните случаи включват контролните вектори на MD5, SHA-1, SHA-256, HMAC-SHA-256, PBKDF2 и пълния обмен SCRAM-SHA-256 от RFC 7677 [17]; всеки от тях съвпада с публикуваната стойност. Интеграционните случаи проверяват, че записана стойност от тип `numeric`, `timestamp` и масив се прочита обратно със същата стойност през двоичния формат.

6.2. Закъснителност на едно заявление

Таблица 6. Закъснителност на едно логическо действие, PostgreSQL 16.4, един ред, микросекунди

Вид заявление	Медиана	95-и проц.	Средно	Ст. откл.	n
Прост цикъл	2114	8149	3197	3589	500
Разширен цикъл, празен кеш	3840	15950	5711	5589	500
Разширен цикъл, попадение в кеша	1657	5043	2178	1579	500

Таблица 7. Закъснителност на едно логическо действие, MySQL 8.0.39, текстов протокол, микросекунди

Вид заявление	Медиана	95-и проц.	Средно	Ст. откл.	n
Един ред	938	10308	2744	4668	500
5000 реда	9409	22370	13310	11397	100

Данните за Табл. 6 и Табл. 7 са от един пуск на `build/benchmarks/conduit_bench --raw` при вдигната среда от `docker-compose.yml`, на машината от Табл. 4.

Разсейването е голямо и това е най-важното наблюдение за самите числа: стандартното отклонение е от порядъка на средната стойност, а 95-и процентил е три до четири пъти над медианата. При такова разпределение сравнение на средни стойности не носи информация. Причината е в средата: сървърите работят в контейнери на същата машина, а не на отделен хост, и планировчикът на виртуализационния слой внася закъснения, които не зависят от клиента. Затова изводите по-долу се правят само там, където разликата надхвърля разсейването, и се подкрепят от небремеви показател.

6.3. Обръщания по мрежата и брой съобщения

Таблица 8. Брой протоколни съобщения за едно логическо действие, PostgreSQL

Действие	Съобщения	Обръщания	Медиана, μs
Прост цикъл	5	1	2114
Разширен цикъл, празен кеш	15	1	3840
Разширен цикъл, попадение в кеша	7	1	1657

Броят съобщения е измерен от вградения механизъм за проследяване и е записан в същия пуск. Той е показателят, при който кешът има най-ясен ефект и който не зависи от машината. Наблюденията са три.

Първо, и трите случая струват едно обръщане по мрежата. Разширеният цикъл *не* плаща допълнително обръщане при първото изпълнение, защото `Parse`, `Describe`, `Bind`, `Execute` и `Sync` се записват в един буфер и се изпращат заедно. Хипотезата, че разширеният цикъл струва две обръщания при първо изпълнение, е опровергана от собствената реализация: тя би била вярна само за клиент, който изпраща `Flush` и чака

отговор след Parse.

Второ, кешът намалява броя съобщения от 15 на 7, тоест с 53 на сто. Спестени са пет изпратени и три получени съобщения: Parse и Describe от клиента, ParseComplete, ParameterDescription и RowDescription от сървъра.

Трето, ефектът върху времето е в същата посока и надхвърля разсейването: медианата пада от 3840 на 1657 микросекунди, тоест с 57 на сто, а стандартното отклонение също пада от 5589 на 1579. Заявление с попадение в кеша е по-бързо и от простия цикъл, 1657 срещу 2114 микросекунди по медиана, което е очаквано: то изпраща по-малко байтове и не кара сървъра да разбира текста на заявлението наново.

6.4. Текстов срещу двоичен формат

Таблица 9. Голям резултат от 5000 реда и четири колони, PostgreSQL, микросекунди

Формат	Медиана	95-и проц.	Средно	Ст. откл.	Декодиране
Текстов	11263	27935	14306	9334	2330
Двоичен	7104	17072	8910	4674	280

Колоната „Декодиране“ е времето, прекарано само в декодиращите функции за целия резултат, измерено с отделен часовник в обработчика на ред и осреднено по повторенията. Разликата е 8,3 пъти: 2330 срещу 280 микросекунди за 5000 реда по три декодирани колони, тоест 155 срещу 19 наносекунди на стойност. Причината е конкретна и е в кода: двоичното декодиране на цяло число е едно зареждане и разместване на байтове, докато текстовото минава през разбор на десетичен запис, а двоичното декодиране на double е `std::bit_cast`, докато текстовото вика `strtod`.

Общото време също е по-малко за двоичния формат, 7104 срещу 11263 микросекунди по медиана. Разликата от 4159 микросекунди е по-голяма от разликата в декодирането, което е 2050 микросекунди. Остатъкът се обяснява с обема на предадените байтове: двоичното представяне на double е винаги 8 байта, докато текстовото за стойности като 7501.5 е шест или повече знака, а на bool е един байт срещу един знак. Тоест печалбата има два независими източника и по-големият от тях не е декодирането.

6.5. Закъснителност на заемане от пул

Таблица 10. Закъснителност на заемане на връзка, медиана и 95-и процентил в микросекунди

Размер	Едновременно.	Медиана	95-и проц.	Ст. откл.	Чакали
1	1	3,3	6,7	3,8	0 от 40
1	4	3688	6887	2454	159 от 160
1	16	13518	16567	2154	639 от 640
2	1	1,6	2,7	0,7	0 от 40
2	4	2184	7221	2013	158 от 160
2	16	7549	9459	1583	638 от 640
4	1	1,0	1,5	0,4	0 от 40
4	4	0,5	1,5	0,8	0 от 160
4	16	3093	3875	743	636 от 640
8	1	0,5	1,1	0,2	0 от 40
8	4	0,5	1,2	0,5	0 от 160
8	16	929	1363	351	632 от 640

Таблицата се чете на два реда. Когато едновременността не надхвърля размера на пула, заемането струва под две микросекунди и нито едно заемане не изчаква: съпрограмата намира свободна връзка и не се спира изобщо. Когато едновременността надхвърли размера, заемането се превръща в чакане на опашка и цената му е времето за изпълнение на чуждо заявление. При едновременност 16 медианата пада от 13518 микросекунди при пул от една връзка до 929 при пул от осем. Осемкратното увеличение на пула намалява чакането 14,5 пъти, тоест повече от пропорционално: освен че опашката се обслужва от повече връзки, при по-голям пул по-малка част от заявките изобщо стигат до опашката, което личи и от последната колона.

Стандартното отклонение при насищане е около една трета от медианата или по-малко, за разлика от таблиците по-горе, където то е от порядъка на средната стойност. Това е независимо потвърждение, че голямото разсейване при единичните заявления идва от средата, а не от клиента: тук се измерва чакане, породено от самия пул, и то е равномерно.

6.6. Анализ

Разминаванията, които измерването откри, се отнасят към три различни източника.

Разлика в броя копия. Двоичният формат печели 8,3 пъти в декодирането спрямо текстовия. Тази разлика е изцяло в клиента и е точно това, което проектът твърдеше, че прякото говорене по протокол ще направи наблюдаемо: цената е един ред в Табл. 9, а не скрито поведение в чужда библиотека.

Разлика в броя съобщения. Кешът на подготвените заявления намалява съобщенията за едно действие с 53 на сто и медианата с 57 на сто. И двете числа са измерени, а съотношението между тях е достатъчно близко, за да подкрепи обяснението, че печалбата идва именно от спестените съобщения, а не от нещо друго.

Разлика, дължаща се на средата. Всичко останало. Разсейването при единичните заявления е от порядъка на средната стойност и не е свойство на клиента: същият клиент, измерен по показател, който не минава през контейнера, дава стабилни числа.

Едно очакване е опровергано от измерването: разширеният цикъл не струва допълнително обръщане по мрежата при първо изпълнение, защото петте съобщения се изпращат заедно. Той струва повече *байтове* и повече работа на сървъра, което личи от медианата, но не и повече обръщания.

Едно сравнение липсва изцяло: с `libpq` и `libmysqlclient`. Те не са налични на машината, измервателната програма го съобщава и не отчита число. Твърдение за отношението към доставчиковите библиотеки не се прави. **[TODO: сравнение с `libpq` и `libmysqlclient` на машина, на която те са инсталирани]**

[TODO: измерване на върховата потребена памет; програмата не го отчита и затова колона за памет липсва във всички таблици по-горе]

[TODO: измерване на пропускливост при фиксиран брой едновременни връзки; измерена е закъснителността на заемане, но не и броят завършени заявления за единица време]

ЗАКЛЮЧЕНИЕ

Проектът разработва клиентска библиотека на C++20, която реализира пряко мрежовите протоколи на PostgreSQL и MySQL, с асинхронен пул от връзки, кеш на подготвените заявления и декодиране на редове без копие. Изборът да се говори по протокола, вместо да се използва слоест диспечер на драйвери, е направен съзнателно и е обоснован в Раздел II с това, че прави цената на достъпа наблюдаема.

Предимства на решението. Липсата на междинни слоеве премахва копия, които иначе са скрити, и измерването го потвърждава: декодирането от двоичен формат е 8,3 пъти по-евтино от текстовото, а числото е получено с часовник вътре в собствения обработчик на ред, тоест точно на мястото, където би стоял чуждият код. Асинхронният модел е естествен за C++20 и не изисква нишка на връзка: цялата библиотека, целият пул и всички измервания се изпълняват в един цикъл на събитията върху една нишка, без нито един ключ за заключване. Типовото изображение се проверява при компилация. Отделно предимство, което не беше предвидено, е че проектът се сглобява на чиста машина само с компилатор и CMake: няма нито една задължителна външна зависимост.

Недостатъци и ограничения. Проектът поема поддръжката на две независими протоколни реализации и носи риска от разминаване при нови версии на сървърите. Обхватът на протоколите не е пълен и това е съзнателно ограничение. Извън обхвата остават: за PostgreSQL режимът COPY, известията LISTEN/NOTIFY, отмяната на изпълняващо се заявление по втора връзка, шифроването с TLS и удостоверяването с GSSAPI или SSPI; за MySQL двоичният протокол за подготвени заявления, компресията, приставката caching_sha2_password и отново TLS. Декодирането обхваща едномерни масиви, а не вложени. Липсата на TLS означава, че библиотеката е годна за връзка в доверена мрежа и не е годна за връзка през публична. Декодирането без копие прехвърля отговорност за живота на данните върху приложението и това е записано в публичния договор, а не подразбиращо се.

Инженерна трактовка на резултатите. Прякото говорене по протокол дава измерима полза на две места. Първото е изборът на формат: двоичният е 8,3 пъти по-евтин за декодиране и общо 37 на сто по-бърз за резултат от 5000 реда, а изборът е достъпен само защото клиентът сам изпраща Bind и сам решава какъв формат да поиска. Второто е кешът на подготвените заявления: 53 на сто по-малко протоколни съобщения и 57 на сто

по-ниска медиана. И двете разлики надхвърлят разсейването.

На едно място измерването опроверга очакване, което изглеждаше очевидно: разширеният цикъл *не* струва допълнително обръщане по мрежата при първо изпълнение, защото петте съобщения се изпращат в един запис. На още едно място измерването откри дефект, който тестовете не бяха уловили: закъснителността на заемане от пул излезе двумодална, което издаде, че освободената връзка се връща в общия списък и бива взета обратно от същата съпрограма, преди събуденият чакащ да се изпълни. Поправката е пряко предаване на връзката и е покрита с тест. Това е и най-полезният резултат от целия експеримент: измерването намери грешка в поведение, което изглеждаше правилно.

Сравнение с `libpq` и `libmysqlclient` не е направено, защото те не са налични на машината, на която са извършени измерванията. Никакво твърдение за отношението към тях не се прави.

Посоки за по-нататъшна работа. Естествените продължения са четири: сравнението с доставчиковите библиотеки на машина, на която те са инсталирани, тъй като клонът в измервателната програма вече съществува и чака само средата; разширяване на покритието на протоколите до частите, изброени по-горе, като TLS е първата по важност; добавяне на трети протокол, което би проверило доколко общият слой наистина е общ; и слой за съвместимост с интерфейс от типа на CLI [4], който би позволил библиотеката да се използва от код, писан срещу стандартния интерфейс.

ЛИТЕРАТУРА

- [1] PostgreSQL Global Development Group, “Frontend/Backend Protocol.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/protocol.html>. **[TODO: дата на достъп]**.
- [2] Oracle Corporation, “MySQL Client/Server Protocol.” MySQL Internals Manual, https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_PROTOCOL.html. **[TODO: дата на достъп]**.
- [3] International Organization for Standardization, “Information technology, Database languages, SQL, Part 1: Framework (SQL/Framework),” Tech. Rep. ISO/IEC 9075-1:2023, ISO/IEC, 2023.
- [4] International Organization for Standardization, “Information technology, Database languages, SQL, Part 3: Call-Level Interface (SQL/CLI),” Tech. Rep. ISO/IEC 9075-3:2023, ISO/IEC, 2023.
- [5] Microsoft Corporation, “ODBC Programmer’s Reference.” Microsoft Learn, <https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-programmer-s-reference>. **[TODO: дата на достъп]**.
- [6] Microsoft Corporation, “ADO.NET Overview.” Microsoft Learn, <https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview>. **[TODO: дата на достъп]**.
- [7] T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, “Extensible Markup Language (XML) 1.0 (Fifth Edition).” W3C Recommendation, <https://www.w3.org/TR/xml/>, 2008. **[TODO: дата на достъп]**.
- [8] J. Sermersheim, “Lightweight Directory Access Protocol (LDAP): The Protocol,” Tech. Rep. RFC 4511, Internet Engineering Task Force, 2006.
- [9] PostgreSQL Global Development Group, “libpq: C Library.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/libpq.html>. **[TODO: дата на достъп]**.
- [10] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

- [11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [12] International Organization for Standardization, "Programming languages, C++," Tech. Rep. ISO/IEC 14882:2020, ISO/IEC, 2020.
- [13] R. Rivest, "The MD5 Message-Digest Algorithm," Tech. Rep. RFC 1321, Internet Engineering Task Force, 1992.
- [14] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," Tech. Rep. FIPS PUB 180-4, U.S. Department of Commerce, 2015.
- [15] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," Tech. Rep. RFC 2104, Internet Engineering Task Force, 1997.
- [16] C. Newman, A. Menon-Sen, A. Melnikov, and N. Williams, "Salted Challenge Response Authentication Mechanism (SCRAM) SASL and GSS-API Mechanisms," Tech. Rep. RFC 5802, Internet Engineering Task Force, 2010.
- [17] T. Hansen, "SCRAM-SHA-256 and SCRAM-SHA-256-PLUS SASL Mechanisms," Tech. Rep. RFC 7677, Internet Engineering Task Force, 2015.
- [18] Docker, Inc., "Compose file reference." Docker Documentation, <https://docs.docker.com/reference/compose-file/>. **[TODO: дата на достъп]**.

ПРИЛОЖЕНИЯ

Приложение А. Схема на тестовата база

Схемата се създава от самата измервателна програма в началото на всеки пуск, така че началното състояние е част от измерването, а не предположение за него. Таблицата се пълни с 5000 реда, освен ако не е зададено друго с `--rows=N`.

```
1 DROP TABLE IF EXISTS conduit_bench;  
2 CREATE TABLE conduit_bench (  
3     id      bigint primary key,  
4     label   text,  
5     value   double precision,  
6     flag    boolean);  
7 -- 5000 реда, вмъкнати с едно заявление:  
8 -- (1,'label1',1.5,true), (2,'label2',3.0,false), ...
```

Листинг 6: Схема на измервателната таблица, PostgreSQL

```
1 DROP TABLE IF EXISTS conduit_bench;  
2 CREATE TABLE conduit_bench (  
3     id      bigint primary key,  
4     label   varchar(64),  
5     value   double,  
6     flag    tinyint(1));
```

Листинг 7: Схема на измервателната таблица, MySQL

Показната програма и интеграционните тестове ползват втора таблица с по-широк набор от типове, включително `numeric`, `timestamp` и масив, за да бъде проверено декодирането на всеки от тях:

```
1 CREATE TABLE conduit_items (  
2     id      bigint primary key,  
3     label   text,  
4     weight  double precision,  
5     flag    boolean,  
6     amount  numeric(12,3),  
7     made_at timestamp,  
8     tags    int4[]);  
9 INSERT INTO conduit_items VALUES  
10 (1,'alpha',1.5,true, 123456.789,'2024-02-29 13:45:30.123456',{1,2,3}),  
11 (2,'beta', 2.5,false, -0.050,'1999-12-31 23:59:59',{}),  
12 (3,'gamma',3.5,true,  0.001,'2000-01-01 00:00:00',NULL);
```

Листинг 8: Таблица за проверка на декодирането на типовете

Приложение Б. Конфигурация на средата

```
1 # Servers for the integration tests and the demonstration.
2 #
3 # The versions are pinned so a measurement can be repeated later against the
4 # same server. The ports are shifted away from the defaults so the containers
5 # do not collide with a PostgreSQL or MySQL already installed on the machine.
6 #
7 #   docker compose up -d
8 #   ctest --test-dir build -L integration
9 #   docker compose down -v
10
11 services:
12     postgres:
13         image: postgres:16.4
14         container_name: conduit_postgres
15         environment:
16             POSTGRES_USER: conduit
17             POSTGRES_PASSWORD: conduit
18             POSTGRES_DB: conduit
19             # Left at the image default, which is scram-sha-256 on this version.
20 ↪ The
21         # client implements SCRAM, so weakening the server would hide the code
22         # that actually runs against a modern PostgreSQL.
23         POSTGRES_HOST_AUTH_METHOD: scram-sha-256
24     ports:
25         - "55432:5432"
26     healthcheck:
27         test: ["CMD-SHELL", "pg_isready -U conduit -d conduit"]
28         interval: 2s
29         timeout: 3s
30         retries: 30
31
32     mysql:
33         image: mysql:8.0.39
34         container_name: conduit_mysql
35         # mysql_native_password is the plugin this client implements. MySQL 8.0
36         # still ships it; on 8.4 and later it has to be enabled explicitly.
```

```

36     command: --default-authentication-plugin=mysql_native_password
37     environment:
38         MYSQL_ROOT_PASSWORD: conduit_root
39         MYSQL_USER: conduit
40         MYSQL_PASSWORD: conduit
41         MYSQL_DATABASE: conduit
42     ports:
43     - "33306:3306"
44     healthcheck:
45         test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -uconduit -pconduit
↪ --silent"]
46         interval: 3s
47         timeout: 3s
48         retries: 40

```

Листинг 9: `docker-compose.yml`: версиите на сървърите, с които са направени измерванията

Портовете са изместени от подразбиращите се, за да не се сблъскат контейнерите с евентуално вече инсталиран сървър на същата машина. Удостоверяването при PostgreSQL е оставено на подразбиращото се `scram-sha-256`, защото клиентът реализира SCRAM и отслабването на сървъра би скрило кода, който наистина се изпълнява.

Приложение В. Избрани части от изходния текст

```

1 // The one thing the two protocol codecs have in common, written down.
2 //
3 // PostgreSQL and MySQL share no base class and no message vocabulary. What
↪ they
4 // do share is the shape of the framing step: given the bytes received so far
↪ ,
5 // either produce one whole frame or say "not yet", without consuming
↪ anything.
6 // Expressing that as a concept rather than a virtual interface means the
↪ layers
7 // above can be templates and pay nothing for the abstraction.
8 #pragma once
9

```

```

10 #include <concepts>
11 #include <optional>
12
13 #include "conduit/bytes.hpp"
14
15 namespace conduit {
16
17 template <class C>
18 concept frame_codec = requires(byte_span in, const typename C::frame& f) {
19     typename C::frame;
20     // Never consumes. A partial frame is reported, not buffered internally,
21     // which is what lets the same codec be driven by a socket or by a test
22     ↪ that
23     // feeds it one byte at a time.
24     { C::peek(in) } -> std::same_as<std::optional<typename C::frame>>;
25     // Total size of the frame including its header, so the caller knows how
26     // much to drop from the receive buffer.
27     { f.consumed } -> std::convertible_to<std::size_t>;
28     // A human readable name for the trace.
29     { C::describe(f) } -> std::convertible_to<const char*>;
30 };
31 } // namespace conduit

```

Листинг 10: include/conduit/codec.hpp: общото между двата кодека, изразено като ИЗИСКВАНЕ КЪМ ТИПА

```

1 // A least recently used map, used for the prepared statement cache.
2 //
3 // Eviction returns the evicted entry rather than dropping it, because a
4 // prepared statement that leaves the cache still exists on the server and
5 ↪ has
6 // to be closed. A cache that silently forgot the name would leak one server
7 // side statement per eviction.
8 #pragma once
9
10 #include <cstddef>
11 #include <list>

```

```

11 #include <optional>
12 #include <string>
13 #include <unordered_map>
14 #include <utility>
15
16 namespace conduit {
17
18 template <class Value>
19 class lru_cache {
20 public:
21     using entry = std::pair<std::string, Value>;
22
23     explicit lru_cache(std::size_t capacity) : capacity_(capacity) {}
24
25     Value* find(const std::string& key) {
26         auto it = index_.find(key);
27         if (it == index_.end()) { ++misses_; return nullptr; }
28         ++hits_;
29         order_.splice(order_.begin(), order_, it->second);
30         return &it->second->second;
31     }
32
33     // Inserts or replaces. Returns the entry that had to be evicted, if any.
34     std::optional<entry> put(std::string key, Value value) {
35         auto it = index_.find(key);
36         if (it != index_.end()) {
37             it->second->second = std::move(value);
38             order_.splice(order_.begin(), order_, it->second);
39             return std::nullopt;
40         }
41         order_.emplace_front(key, std::move(value));
42         index_.emplace(std::move(key), order_.begin());
43
44         if (capacity_ == 0 || order_.size() <= capacity_) return std::nullopt;
45         ↵ ;
46         entry victim = std::move(order_.back());
47         index_.erase(victim.first);
48         order_.pop_back();
49         ++evictions_;

```

```

49     return victim;
50 }
51
52 std::optional<entry> erase(const std::string& key) {
53     auto it = index_.find(key);
54     if (it == index_.end()) return std::nullopt;
55     entry gone = std::move(*it->second);
56     order_.erase(it->second);
57     index_.erase(it);
58     return gone;
59 }
60
61 // Drains the cache, handing every entry back so the caller can close
↪ each
62 // one on the server.
63 std::list<entry> take_all() {
64     index_.clear();
65     std::list<entry> out;
66     out.swap(order_);
67     return out;
68 }
69
70 std::size_t size() const noexcept { return order_.size(); }
71 std::size_t capacity() const noexcept { return capacity_; }
72 std::size_t hits() const noexcept { return hits_; }
73 std::size_t misses() const noexcept { return misses_; }
74 std::size_t evictions() const noexcept { return evictions_; }
75 void reset_stats() noexcept { hits_ = misses_ = evictions_ = 0; }
76
77 // Most recently used first. Only the tests need the order made visible.
78 std::vector<std::string> keys_in_order() const {
79     std::vector<std::string> out;
80     out.reserve(order_.size());
81     for (const auto& e : order_) out.push_back(e.first);
82     return out;
83 }
84
85 private:
86     std::size_t capacity_;

```

```

87     std::list<entry> order_;
88     std::unordered_map<std::string, typename std::list<entry>::iterator>
↪ index_;
89     std::size_t hits_ = 0, misses_ = 0, evictions_ = 0;
90 };
91
92 } // namespace conduit

```

Листинг 11: include/conduit/lru_cache.hpp: кешът на подготвените заявления; измественият запис се връща на извикващия, за да бъде затворен на сървъра

```

1 // Typed row mapping with compile time field binding.
2 //
3 // The user declares which struct member each column feeds. The binding is a
4 // constexpr tuple of member pointers, so the decoder chosen for every field
↪ is
5 // picked at compile time and a member whose type has no field_codec is a
6 // compilation error, not a run time surprise.
7 //
8 // Column names are resolved once per result set, not once per row: the
↪ mapper
9 // stores the resolved indices.
10 //
11 //     struct book { std::int64_t id; std::string title; double price; };
12 //     template <> struct conduit::row_mapping<book> {
13 //         static constexpr auto fields = std::tuple{
14 //             conduit::bind_field("id", &book::id),
15 //             conduit::bind_field("title", &book::title),
16 //             conduit::bind_field("price", &book::price)};
17 //     };
18 #pragma once
19
20 #include <array>
21 #include <cstdint>
22 #include <span>
23 #include <stdexcept>
24 #include <string>
25 #include <string_view>

```

```

26 #include <tuple>
27 #include <type_traits>
28 #include <utility>
29 #include <vector>
30
31 #include "conduit/value.hpp"
32
33 namespace conduit {
34
35 class mapping_error : public std::runtime_error {
36 public:
37     explicit mapping_error(const std::string& what) : std::runtime_error(what
↪ ) {}
38 };
39
40 template <class Struct, class Member>
41 struct field_binding {
42     const char* name;
43     Member Struct::* pointer;
44     using member_type = Member;
45     using struct_type = Struct;
46 };
47
48 template <class Struct, class Member>
49 constexpr field_binding<Struct, Member> bind_field(const char* name, Member
↪ Struct::* p) {
50     return {name, p};
51 }
52
53 // Specialise this for every struct that is read from a result set.
54 template <class T>
55 struct row_mapping;
56
57 template <class T>
58 concept mappable = requires { row_mapping<T>::fields; };
59
60 template <mappable T>
61 class row_mapper {
62 public:

```

```

63     static constexpr auto bindings = row_mapping<T>::fields;
64     static constexpr std::size_t field_count =
65         std::tuple_size_v<std::remove_cvref_t<decltype(bindings)>>>;
66
67     // 'columns' is any range whose elements have a '.name' convertible to a
68     // string view. Both backends satisfy that without sharing a base class,
69     // which is the point of the design.
70     template <class Columns>
71     explicit row_mapper(const Columns& columns) {
72         [&]<std::size_t... I>(std::index_sequence<I...>) {
73             ((indices_[I] = resolve(columns, std::get<I>(bindings).name)),
↪ ...);
74             }(std::make_index_sequence<field_count>{});
75     }
76
77     T operator()(std::span<const field_view> row) const {
78         T out{};
79         [&]<std::size_t... I>(std::index_sequence<I...>) {
80             (assign<I>(out, row), ...);
81             }(std::make_index_sequence<field_count>{});
82         return out;
83     }
84
85     std::size_t index_of(std::size_t field) const { return indices_[field]; }
86
87 private:
88     template <std::size_t I>
89     void assign(T& out, std::span<const field_view> row) const {
90         constexpr auto binding = std::get<I>(bindings);
91         using member = typename std::remove_cvref_t<decltype(binding)>::
↪ member_type;
92         static_assert(requires(field_view f) { field_codec<member>::decode(f)
↪ ; },
93
94             "no field_codec for this member type; add a
↪ specialisation or "
95
96             "change the member to a supported type");
97         if (indices_[I] >= row.size())
98             throw mapping_error(std::string("column ") + binding.name + " is
↪ not in the row");

```



```

97         out.*(binding.pointer) = field_codec<member>::decode(row[indices_[I
↪ ]]);
98     }
99
100     template <class Columns>
101     static std::size_t resolve(const Columns& columns, std::string_view name)
↪ {
102         std::size_t i = 0;
103         for (const auto& c : columns) {
104             if (std::string_view(c.name) == name) return i;
105             ++i;
106         }
107         throw mapping_error("result set has no column named " + std::string(
↪ name));
108     }
109
110     std::array<std::size_t, field_count> indices_{};
111 };
112
113 } // namespace conduit

```

Листинг 12: include/conduit/row_map.hpp: типовото изображение със свързване на полетата при компилация

Кодекът на съобщенията за PostgreSQL е в src/pg_message.cpp и include/conduit/pg/protocol.hpp; кодекът на пакетите за MySQL е в src/mysql_packet.cpp и include/conduit/mysql/protocol.hpp; асинхронният пул е в include/conduit/pool.hpp. Ключовите места от всеки от тях са показани в Раздел IV.

Приложение Г. Пълни резултати от измерванията

Суровите проби от всички повторения се записват от измервателната програма при подаден флаг --raw. Пускът, от който са изведени таблиците в Раздел VI, произведе 5660 записа в benchmarks/results/raw_samples.csv. Форматът е един ред на повторение:

```

1 case,repetition,microseconds
2 pg_simple_one_row,0,3075.500

```

```

3 pg_simple_one_row,1,1468.600
4 pg_simple_one_row,2,1063.400
5 pg_simple_one_row,3,983.700
6 pg_simple_one_row,4,2477.800
7 ...

```

Листинг 13: Начало на файла със суровите проби

Имената на случаите във файла са: `pg_simple_one_row`, `pg_extended_first_execution`, `pg_extended_cached`, `pg_large_result_text`, `pg_large_result_binary`, `mysql_simple_one_row`, `mysql_large_result_text` и по един `pool_acquire_sizeN_concM` за всяка от дванадесетте комбинации размер и едновременност.

```

1 Conduit measurements
2   repetitions 500, warm up repetitions 20 (discarded), large result 5000 rows
3
4 PostgreSQL, one row          median      p95      mean    stddev    n
5   simple query flow          2114.0    8148.6    3197.3    3589.3    500
6   extended flow, first execution 3839.7    15950.3    5711.3    5589.4    500
7   extended flow, cached      1656.5     5043.1    2177.6    1578.7    500
8
9 PostgreSQL, 5000 rows
10  text results                11262.7    27935.1    14305.9    9333.8    100
11  binary results              7103.8     17071.8    8909.9     4673.8    100
12
13 Protocol messages per logical query
14   simple query flow          5
15   extended flow, first execution 15
16   extended flow, statement cached 7
17
18 MySQL
19   text protocol, one row      937.7     10308.5    2743.9     4667.5    500
20   text protocol, 5000 rows    9408.6    22370.4    13309.8    11396.7    100
21
22 libpq baseline: not built, libpq was not found on this machine

```

Листинг 14: Обобщен изход на измервателната програма за пуска, от който са таблиците в Раздел VI